

## Rapport d'Analyse HTTP

Projet Wireshark Date : 01 Juin 2026

Protocole : HTTP (HyperText Transfer Protocol)

Outil : Wireshark sur Kali Linux

### Introduction : Protocole HTTP et generation du trafic

Dans cet exercice j'ai analyse le protocole HTTP en capturant du trafic reseau reel genere depuis mon terminal Kali Linux.

Pour creer du trafic HTTP pur sans chiffrement, j'ai utilise trois commandes curl vers des sites qui n'utilisent pas HTTPS :

```
curl http://example.com
```

```
curl http://neverssl.com
```

```
curl http://httpforever.com
```

Le filtre utilise dans Wireshark etait http, ce qui affiche uniquement les echanges applicatifs HTTP sans les paquets TCP sous-jacents.

Ces trois commandes ont genere six paquets HTTP analyses, une requete GET et une reponse 200 OK pour chaque site. L'objectif de ce module est d'observer comment HTTP transporte les donnees en clair, quels headers sont visibles sur le reseau, et pourquoi HTTPS est indispensable pour securiser ces echanges.

### Le flux complet TCP et HTTP

Ce que Wireshark montre reellement avant chaque requete HTTP :

Paquet 28 -> TCP SYN            mon Kali envoie une demande de connexion

Paquet 29 -> TCP SYN-ACK       le serveur accepte

Paquet 30 -> TCP ACK           connexion TCP etablie

Paquet 31 -> HTTP GET /        requete HTTP enfin envoyee

Paquet 35 -> HTTP 200 OK       reponse du serveur

Paquet 36 -> TCP FIN ACK       fermeture de la connexion

HTTP ne peut pas exister sans ce handshake TCP avant lui. Le filtre http dans Wireshark cache ces paquets TCP mais ils existent toujours en dessous.

## Paquet 31 - Requete GET example.com

### 1. C'est quoi HTTP

HTTP est le protocole qui permet a un navigateur ou a curl de demander des pages web a un serveur. Il fonctionne en mode question-reponse : le client envoie une requete, le serveur repond. Tout ce qui circule est en clair, lisible par n'importe qui sur le reseau. C'est pour ca que HTTPS existe.

### 2. Ce que je vois dans ce paquet

Mon Kali avec l'adresse 192.168.43.138 envoie une requete vers le serveur example.com qui a l'adresse 172.66.147.243.

Le port source est 39574, genere aleatoirement par Linux pour cette session.

Le port destination est 80, c'est le port standard de HTTP sur tous les serveurs.

La trame complete fait 141 octets.

Le payload HTTP, c'est-a-dire la requete pure, fait 75 octets de texte lisible en clair.

### 3. Ce que les headers m'apprennent

GET / signifie que je demande la page principale du site, la racine.

HTTP/1.1 est la version du protocole qui garde la connexion TCP ouverte par default entre plusieurs requetes.

Host: example.com indique au serveur quel site je veux, utile quand plusieurs sites partagent la meme IP.

User-Agent: curl/8.15.0 revele exactement l'outil que j'utilise et sa version.

Accept: indique que j'accepte n'importe quel type de contenu en reponse.

### 4. Point securite

Le header User-Agent expose curl/8.15.0 en clair sur le reseau. Un attaquant qui intercepte ce paquet sait immediatement que j'utilise curl version 8.15.0 sur Linux. Il peut chercher des vulnerabilites connues de cette version. En pentest, falsifier ce header est une technique courante pour se faire passer pour un navigateur normal.

### 5. HTTP vs HTTPS

En HTTP, cette requete avec tous ses headers est lisible mot pour mot par n'importe quel equipement sur le chemin. En HTTPS, des que le handshake TLS est termine, tout ce contenu devient un bloc chiffre illisible. Le port destination passe de 80 a 443.

## Paquet 35 - Reponse 200 OK example.com

### 1. Les codes de reponse HTTP

Quand le serveur recoit une requete GET, il repond avec un code qui indique si ca a marche ou non. Les codes 2xx signifient succes. Les codes 4xx signifient erreur cote client. Les codes 5xx signifient erreur cote serveur. Le 200 OK est le code de succes standard.

### 2. Ce que je vois dans ce paquet

Le serveur example.com avec l'adresse 172.66.147.243 me repond.

Le port source est 80 vers mon port 39574, retour vers le meme port que j'avais utilise.

Les headers de reponse font 71 octets.

Le contenu HTML complet fait 528 octets, c'est le code de la page web envoye en clair.

La trame totale est donc plus grande que 71 octets car elle contient les headers plus le HTML.

### 3. Ce que les headers m'apprennent

HTTP/1.1 200 OK confirme que la requete a reussi et voici la page demandee.

Server: cloudflare indique que example.com passe par l'infrastructure Cloudflare comme reverse proxy.

Content-Type: text/html; charset=UTF-8 signifie que le serveur m'envoie du HTML encode en UTF-8.

Last-Modified: Thu, 28 May 2026 04:54:11 GMT indique la date de derniere modification de la ressource.

### 4. Point securite

Le header Server: cloudflare revele qu'un reverse proxy est en place. Cloudflare masque volontairement la version exacte du serveur reel derriere son infrastructure. C'est une bonne pratique de securite. Le header Last-Modified revele aussi une date precise qui peut aider un attaquant a cartographier l'infrastructure sans meme toucher au serveur.

### 5. HTTP vs HTTPS

En HTTP, le code HTML complet de la page est visible dans Wireshark sous l'onglet Line-based text data. Je peux lire chaque ligne de code. En HTTPS, Wireshark affiche uniquement Application Data, un bloc binaire chiffre totalement illisible.

Paquet 53 - Requete GET neverssl.com

### 1. L'encapsulation HTTP sur TCP

HTTP ne voyage pas seul sur le reseau. Il s'appuie sur TCP pour le transport. Avant que ce paquet GET parte, un handshake TCP complet a eu lieu avec SYN, SYN-ACK et ACK. HTTP utilise TCP parce qu'il a besoin de la fiabilite, une page web doit arriver complete et dans le bon ordre.

### 2. Ce que je vois dans ce paquet

Mon Kali avec l'adresse 192.168.43.138 envoie une requete vers neverssl.com qui a l'adresse 34.223.124.45.

Le port source est 60848, Linux ouvre un nouveau port pour chaque connexion TCP.

Le port destination est 80.

La trame complete fait 142 octets.

Les flags TCP actifs sont PSH et ACK avec la valeur 0x018.

### 3. Ce que les headers m'apprennent

GET / HTTP/1.1 est la demande de la page principale.

Host: neverssl.com est le nom du site demande, visible en clair.

User-Agent: curl/8.15.0 revele toujours le meme outil utilise.

Accept: indique que j'accepte n'importe quel type de contenu en reponse.

Le flag TCP PSH force le systeme a envoyer immediatement les donnees sans attendre que le buffer soit plein.

Le flag TCP ACK confirme la reception des donnees precedentes.

### 4. Point securite

Le header Host: neverssl.com est visible en clair. Sur un reseau HTTP, n'importe qui peut voir exactement quel site tu visites. Meme avec HTTPS, le nom de domaine reste partiellement visible via le SNI (Server Name Indication) dans le handshake TLS, mais le chemin et les donnees sont chiffres.

### 5. HTTP vs HTTPS

En HTTP, le nom de domaine dans le header Host est lisible par tous. En HTTPS, bien que le SNI revele encore le domaine pendant le handshake, tout ce qui vient apres est chiffre et le contenu de la requete est totalement protege.

Paquet 55 - Reponse 200 OK neverssl.com

### 1. Le danger du contenu HTML en clair

Quand un serveur repond en HTTP, il envoie le code HTML de la page directement sur le reseau sans aucune protection. N'importe qui entre toi et le serveur peut lire ce contenu, le modifier, ou y injecter du code malveillant avant qu'il n'arrive sur ton ecran.

### 2. Ce que je vois dans ce paquet

Le serveur neverssl.com avec l'adresse 34.223.124.45 me repond.

Le port source est 80 vers mon port 60848.

La trame fait 4327 octets, beaucoup plus grande que example.com car la page contient du CSS.

Le payload textuel fait 3961 octets de code HTML et CSS lisible en clair.

### 3. Ce que les headers m'apprennent

HTTP/1.1 200 OK confirme le succes.

Server: Apache/2.4.66 revele le serveur web et sa version exacte.

Content-Type: text/html; charset=UTF-8 indique du contenu HTML encode en UTF-8.

Last-Modified: Wed, 29 Jun 2022 00:23:33 GMT indique la date de derniere modification.

La balise title NeverSSL - Connecting anything est visible directement dans Wireshark.

### 4. Point securite

Apache/2.4.66 est visible en clair. Un attaquant peut aller immediatement sur la base de donnees CVE sur [cve.mitre.org](https://cve.mitre.org) et chercher toutes les vulnerabilites connues d'Apache 2.4.66. Si une faille existe et que le serveur n'est pas patche, l'attaquant a un vecteur d'attaque direct. C'est ce qu'on appelle le banner grabbing, c'est-a-dire recuperer les informations d'identification d'un serveur sans meme l'attaquer.

### 5. HTTP vs HTTPS

En HTTP, le code HTML complet est lisible dans Wireshark et modifiable par un attaquant MITM avant d'arriver au client. En HTTPS, Wireshark ne voit qu'un bloc opaque Application Data. Toute modification du contenu invalide le certificat TLS et coupe la connexion.

## Paquet 67 - Requete GET httpforever.com

### 1. HTTP/1.1 et les connexions persistantes

HTTP/1.1 introduit le keep-alive par default. Cela signifie que la connexion TCP reste ouverte apres la premiere requete pour permettre d'autres echanges sans refaire un handshake TCP complet. C'est une optimisation importante pour les pages qui chargent plusieurs ressources comme des images, du CSS ou des scripts.

### 2. Ce que je vois dans ce paquet

Mon Kali avec l'adresse 192.168.43.138 envoie une requete vers httpforever.com qui a l'adresse 146.190.62.39.

Le port source est 39380.

Le port destination est 80.

La trame complete fait 145 octets.

La taille de fenetre TCP est 64512 octets, ce qui indique la quantite de donnees que mon systeme peut recevoir avant d'envoyer un accuse de reception.

### 3. Ce que les headers m'apprennent

GET / HTTP/1.1 est la demande de la page principale.

Host: httpforever.com est le site cible, visible en clair.

User-Agent: curl/8.15.0 revele toujours le meme outil.

Connection: keep-alive demande au serveur de garder la connexion TCP ouverte apres la reponse.

Accept: indique que j'accepte tout type de contenu.

### 4. Point securite

Si cette requete avait contenu un cookie de session ou un token d'authentification comme Authorization: Bearer, ce secret serait visible en clair dans Wireshark. N'importe qui sur le meme reseau WiFi pourrait copier ce token et usurper ton identite sur le site. C'est ce qu'on appelle le session hijacking. C'est pourquoi les sites qui gerent des connexions utilisent obligatoirement HTTPS.

### 5. HTTP vs HTTPS

En HTTP, les cookies et tokens d'authentification transitent en clair. En HTTPS, ces informations sont chiffrees dans le tunnel TLS. Meme si quelqu'un capture le paquet, il ne peut pas lire le cookie sans la cle de dechiffrement.

Paquet 71 - Reponse 200 OK httpforever.com

### 1. Les headers de securite HTTP

Les serveurs peuvent envoyer des headers speciaux pour instruire le navigateur sur les regles de securite a appliquer. Ces headers sont visibles en clair dans Wireshark et permettent d'analyser la posture de securite d'un serveur sans meme l'attaquer.

### 2. Ce que je vois dans ce paquet

Le serveur httpforever.com avec l'adresse 146.190.62.39 me repond.

Le port source est 80 vers mon port 39380.

La trame fait 462 octets pour les headers de reponse.

Le volume total reassemble par TCP sur plusieurs segments fait 5124 octets de code source HTML.

Note importante : les 5124 octets ne viennent pas de ce seul paquet. TCP decoupe les grandes reponses en plusieurs segments et Wireshark les reassemble pour afficher le total. Le paquet 71 seul fait 462 octets.

### 3. Ce que les headers m'apprennent

HTTP/1.1 200 OK confirme le succes.

Server: nginx/1.18.0 (Ubuntu) revele le serveur Nginx, sa version et le systeme d'exploitation Ubuntu.

Content-Security-Policy est une regle qui indique au navigateur quels scripts il est autorise a executer.

Content-Type: text/html indique du contenu HTML.

Last-Modified: Wed, 22 Mar 2023 14:54:48 GMT indique la date de derniere modification.

### 4. Point securite

nginx/1.18.0 (Ubuntu) est la divulgation la plus importante parmi les trois sites analyses. Elle revele en une seule ligne le serveur web, sa version et le systeme d'exploitation utilise.

Ces informations permettent a un attaquant de rechercher d'eventuelles vulnerabilites connues associees a cette version. C'est ce qu'on appelle le server banner grabbing.

La bonne pratique consiste a masquer ou limiter les informations revelees par le header Server afin de reduire les informations disponibles pour un attaquant.

### 5. HTTP vs HTTPS

En HTTP, les headers de securite comme Content-Security-Policy sont lisibles mais aussi modifiables par un attaquant MITM avant d'arriver au navigateur. Un attaquant peut supprimer la CSP et injecter des scripts malveillants. En HTTPS, l'integrite de chaque paquet est garantie par une signature cryptographique. Toute modification invalide la session TLS immediatement.

- Headers HTTP essentiels observes dans cette capture

Headers de requete visibles en clair :

GET /                    methode et ressource demandee  
 Host                    nom de domaine cible  
 User-Agent            outil ou navigateur utilise  
 Accept                formats acceptes en reponse  
 Connection            keep-alive ou close

Headers de reponse visibles en clair :

HTTP/1.1 200        code de statut  
 Server                logiciel serveur, dangereux si version revelee  
 Content-Type        type de contenu renvoye  
 Content-Length      taille du corps de reponse  
 Last-Modified      date de modification de la ressource

- Comparatif des trois sites

| Site            | IP             | Serveur                | Taille | Risque     |
|-----------------|----------------|------------------------|--------|------------|
| example.com     | 172.66.147.243 | cloudflare             | 528 o  | Faible     |
| neverssl.com    | 34.223.124.45  | Apache/2.4.66          | 3961 o | Eleve      |
| httpforever.com | 146.190.62.39  | nginx/1.18.0<br>Ubuntu | 5124 o | Tres eleve |

Cloudflare masque le serveur reel derriere son infrastructure. Apache revele sa version exacte. Nginx revele sa version et meme le systeme d'exploitation. C'est la difference entre bonne et mauvaise configuration de securite.



## Conclusion de ce que j'ai appris sur HTTP

A travers l'analyse des six paquets captures sur trois sites differents, j'ai pu observer concretement pourquoi HTTP seul est considere comme non securise.

La premiere chose evidente est que tout est lisible. La methode GET, l'URL demandee, le User-Agent avec la valeur curl/8.15.0, les headers et le contenu HTML de la reponse sont tous visibles en clair dans Wireshark sans aucun outil special.

Le deuxieme point important est la fuite d'informations cote serveur. neverssl.com revele Apache/2.4.66 et httpforever.com revele nginx/1.18.0 Ubuntu. Ces informations permettent a un attaquant de chercher directement les vulnerabilites connues de ces versions dans les bases CVE. Cloudflare au contraire masque ces details, ce qui est une bonne pratique.

Le troisieme point concerne la relation entre HTTP et TCP. Avant chaque GET HTTP visible dans Wireshark, un handshake TCP complet a eu lieu. HTTP ne peut pas fonctionner sans TCP en dessous. C'est l'encapsulation des protocoles.

La conclusion principale est simple. HTTP transporte tout en clair. Un attaquant sur le meme reseau WiFi peut voir exactement ce que tu demandes et ce que le serveur te repond. HTTPS avec TLS resout ce probleme en chiffrant tout ce trafic des la fin du handshake TCP. Cette analyse m'a permis de comprendre pourquoi HTTPS est devenu le standard sur Internet moderne.